==========

==========

CHAPTER 28 — FULL SYSTEM CASE STUDIES

==========

==========

A maximum-detail, RTOS/QNX-style breakdown of complete MMU designs used in real systems: Linux, QNX, hypervisors, embedded SoCs, automotive ECUs, and safety-critical platforms.

****

****

28.1 — Purpose of Case Studies

****

****

Case studies demonstrate:

- how real systems structure page tables
- how attributes, permissions, and shareability are chosen
- how device memory is mapped safely

- how Stage-1 and Stage-2 interact
- how SMP and DMA constraints shape design
- how safety/security requirements influence layout

These examples are **representative**, not tied to any specific vendor code.

==========

==========

28.2 — Case Study A: Linux Kernel (EL1)

==========

==========

Linux uses a **split address space**:

- **TTBR1_EL1** → kernel
- **TTBR0_EL1** → user space

Kernel Memory Layout

```
[High VA]  Kernel text (1 GB block)
[High VA]  Kernel data (1 GB block)
[High VA]  Modules (2 MB blocks)
[High VA]  Fixmap region (4 KB pages)
[High VA]  Device MMIO (4 KB pages, Device-nGnRnE)
```

User Memory Layout

```
[Low VA]   Code (4 KB pages)
[Low VA]   Data (4 KB pages)
[Low VA]   Heap (4 KB pages)
```

```
[Low VA]   Stack (4 KB pages)
```

Key Design Choices

- Kernel uses **blocks** → low TLB pressure
- User uses **pages** → fine-grained protection
- MMIO uses **Device-nGnRnE**
- Kernel vectors mapped executable + read-only
- PAN + PXN + UXN enforce W^X

Why this works

- high performance
- strong isolation
- safe device access
- scalable SMP behavior

==========

==========

28.3 — Case Study B: QNX Microkernel (Safety-Critical RTOS)

==========

==========

QNX uses **deterministic partitioning**:

Partition Layout

```
Partition A (Safety):
    Code: 2 MB block
    Data: 2 MB block
    MMIO: Device-nGnRnE
    Stack: 4 KB pages

Partition B (Non-Safety):
    Code: 2 MB block
    Data: 2 MB block
    Heap: 4 KB pages

Kernel:
    Global mappings (1 GB blocks)
```

Key Design Choices

- static ASID ranges
- static TTBR1 kernel
- no dynamic remapping
- no overcommit
- deterministic timing
- strict MMIO isolation

Why this works

- ISO 26262 compliance
- predictable behavior
- strong partition isolation

==========

==========

28.4 — Case Study C: KVM/Hypervisor (EL2)

==========

==========

KVM uses **Stage-2 translation** to isolate VMs.

Stage-2 Layout

```
Guest RAM: 2 MB or 1 GB blocks
Guest kernel: 2 MB blocks
Guest user: 4 KB pages
Guest MMIO: 4 KB pages (Device-nGnRnE)
```

Key Design Choices

- VMID tagging
- Stage-2 permissions (S2AP)
- Stage-2 execute control (S2PXN/S2UXN)
- Stage-2 attributes override guest
- GIC virtualization
- timer virtualization

Why this works

- strong VM isolation
- safe device virtualization

- high performance with block mappings

==========

==========

28.5 — Case Study D: Bare-Metal Embedded System

==========

==========

Bare-metal systems use **identity mapping** and **simplicity**.

Memory Layout

```
0x0000_0000 → 0x3FFF_FFFF   RAM (Normal WB)
0x4000_0000 → 0x4FFF_FFFF   MMIO (Device-nGnRnE)
0x5000_0000 → 0x5FFF_FFFF   Flash (Normal WT or WB)
```

Key Design Choices

- single TTBR
- identity mapping
- minimal page tables
- deterministic timing

Why this works

- simplicity
- reliability
- predictable behavior

==========

==========

28.6 — Case Study E: Automotive ECU

==========

==========

Automotive ECUs require **safety + determinism**.

Partition Layout

```
Safety Partition:
    Code: 2 MB block
    Data: 2 MB block
    MMIO: Device-nGnRnE

Non-Safety Partition:
    Code: 2 MB block
    Data: 2 MB block

Kernel:
    Global mappings
```

Key Design Choices

- static partitions
- strict MMIO isolation
- deterministic memory access
- watchdog-safe layout

Why this works

- ISO 26262 compliance
- deterministic timing
- strong isolation

==========

==========

28.7 — Case Study F: Secure Enclave / TEE (EL3 or Secure EL1)

==========

==========

Secure enclaves use **TrustZone**.

Memory Layout

```
Secure World:
    Secure RAM: Normal WB
    Secure MMIO: Device-nGnRnE
    Secure code/data: executable + read-only

Non-Secure World:
    Normal RAM: Normal WB
    MMIO: Device-nGnRnE
```

Key Design Choices

- secure page tables

- secure interrupts
- secure MMIO
- non-secure world cannot access secure memory

Why this works

- strong hardware isolation
- ideal for cryptography and key storage

==========

==========

28.8 — Case Study G: High-Performance Server

==========

==========

Servers use **huge pages** and **NUMA-aware mappings**.

Memory Layout

```
Node 0:
    RAM: 1 GB blocks
Node 1:
    RAM: 1 GB blocks

Kernel:
    1 GB blocks

User:
```

```
2 MB huge pages
```

Key Design Choices

- minimal TLB misses
- minimal page-table walks
- NUMA locality
- high throughput

Why this works

- optimal performance
- scalable SMP behavior

==========

==========

28.9 — Case Study H: GPU/Accelerator-Heavy System

==========

==========

GPU systems require **IOMMU + DMA safety**.

Memory Layout

```
CPU RAM: Normal WB
GPU RAM: Normal WB or Device-nGnRE
GPU MMIO: Device-nGnRnE
```

```
Shared buffers: Normal WB, Inner Shareable
```

Key Design Choices

- IOMMU isolation
- safe DMA buffers
- strict MMIO ordering

Why this works

- high throughput
- safe device access

==========

==========

28.10 — Case Study I: Virtualized Cloud Host

==========

==========

Cloud hosts require **high VM density**.

Memory Layout

```
Host Kernel:
    1 GB blocks

Guest RAM:
    2 MB blocks
```

```
Guest MMIO:
    4 KB pages (Device-nGnRnE)
```

Key Design Choices

- memory overcommit
- ballooning
- live migration
- shadow page tables
- para-virtualization

Why this works

- scalable virtualization
- safe isolation
- efficient migration

==========

==========

28.11 — Case Study J: Robotics / Real-Time Control

==========

==========

Robotics systems require **deterministic latency**.

Memory Layout

```
Control Loop:
    Code: 2 MB block
    Data: 2 MB block

Sensors:
    MMIO: Device-nGnRnE

Actuators:
    MMIO: Device-nGnRnE

Shared Buffers:
    Normal WB, Inner Shareable
```

Key Design Choices

- deterministic memory access
- strict MMIO isolation
- safe DMA buffers

Why this works

- predictable control loops
- safe device access

==========

==========

28.12 — Case Study K: Minimalist Microkernel

==========

==========

Microkernels use **small kernels + user servers**.

Memory Layout

```
Kernel:
    1 GB block

Servers:
    4 KB pages

MMIO:
    Device-nGnRnE
```

Key Design Choices

- minimal kernel footprint
- strong isolation
- simple MMU design

Why this works

- high reliability

- strong security

==========

==========

28.13 — Case Study L: High-Security System

==========

==========

Security-focused systems enforce **strict W^X + PAN + PXN/UXN**.

Memory Layout

```
Code: executable + read-only
Data: writable + non-exec
Stack: writable + non-exec
MMIO: Device-nGnRnE
```

Key Design Choices

- W^X everywhere
- PAN enabled
- PXN/UXN enforced
- Stage-2 overrides guest permissions

Why this works

- strong exploit mitigation
- strict memory separation

==========

==========

28.14 — Case Study M: Multimedia / AV Processing

==========

==========

AV systems require **high bandwidth + low latency**.

Memory Layout

```
Audio buffers: Normal WB
Video buffers: Normal WB
MMIO: Device-nGnRnE
```

Key Design Choices

- safe DMA
- cache-optimized buffers
- strict MMIO ordering

Why this works

- high throughput
- low latency

==========

==========

28.15 — Case Study N: Multi-Tenant Isolation

==========

==========

Multi-tenant systems require **strong isolation**.

Memory Layout

```
Tenant A:
    RAM: 2 MB blocks
Tenant B:
    RAM: 2 MB blocks
Tenant C:
    RAM: 2 MB blocks
```

Key Design Choices

- per-tenant ASID ranges
- per-tenant VMID ranges
- strict Stage-2 isolation
- device virtualization

Why this works

- scalable multi-tenant design
- strong isolation

****

****

28.16 — Chapter Summary

****

****

This chapter covered **complete MMU designs** for:

- Linux
- QNX
- hypervisors
- bare-metal systems
- automotive ECUs
- secure enclaves
- servers
- GPU systems
- cloud hosts
- robotics
- microkernels
- high-security systems
- multimedia systems
- multi-tenant isolation